

Write Up

Table of contents

- [Write Up](#)
 - [Introduction](#)
 - [Similar Programmes](#)
 - [Sandspiel](#)
 - [Overview](#)
 - [likes](#)
 - [dislikes](#)
 - [SandBoxels](#)
 - [Overview](#)
 - [likes](#)
 - [dislikes](#)
 - [Powder Game](#)
 - [overview](#)
 - [likes](#)
 - [dislikes](#)
 - [Stakeholders](#)
 - [STEM students](#)
 - [Secondary school students](#)
 - [Requirements](#)
 - [hardware and software requirements](#)
 - [hardware](#)
 - [software](#)
 - [Limitations](#)
 - [clean ui](#)
 - [logic gates](#)
 - [mechanical](#)
 - [computational methods](#)
 - [heuristics](#)
 - [visualisation](#)
 - [concurrency](#)
 - [UI and UX](#)
 - [Analysis:](#)
 - [Buttons](#)
 - [Background](#)
 - [Font](#)
 - [Analysis](#)
 - [layout](#)

- Development plan
 - UI implementation
 - planning
 - implementation
 - testing
 - element placing
 - element removing and replacing
-

Introduction

I am a computer science and double maths student who has a fascination with elements and small scale details. Because of my interests this project will be a sandbox environment where you can place different coloured pixels that all behave in different ways and interact with each other.

This environment will be made to model a simplified version of reality, for this reason my programme will focus on realistic interactions as this will make it more intuitive to the user.

Similar Programmes

It is important for my project that I analyse previous programmes which have similar functions. by doing this I can avoid mistakes they have made and so I can see features of theirs I like and interpret them in my own.

Sandspiel



<https://sandspiel.club/>

Overview

Sandspiel is the closest example to what I am planning to make. It is a simple sandbox that has few elements, a clean ui and is really easy to start using but also allows for emergent behaviour.

Personally I am a fan of the ui, particle physics, fire, plants and wind which seem simple but have all have similar but fun reactions with each other. Of the features I like, I will try to Implement a similar ui, particle physics and fire and if possible I would implement a more complicated version of the wind they have used. There are also some issues I have with Sandspiel that I would like to avoid in my implementation, most important of which is the "samey" reactions, lots of the elements have which makes lots of the elements blend together whereas I would like to have unique interactions for each elements. some other basic issues I have with Sandspiel is the place and erase tools which I would like to have a lot more precise control of in

my project as this would be useful when making precise interactions.

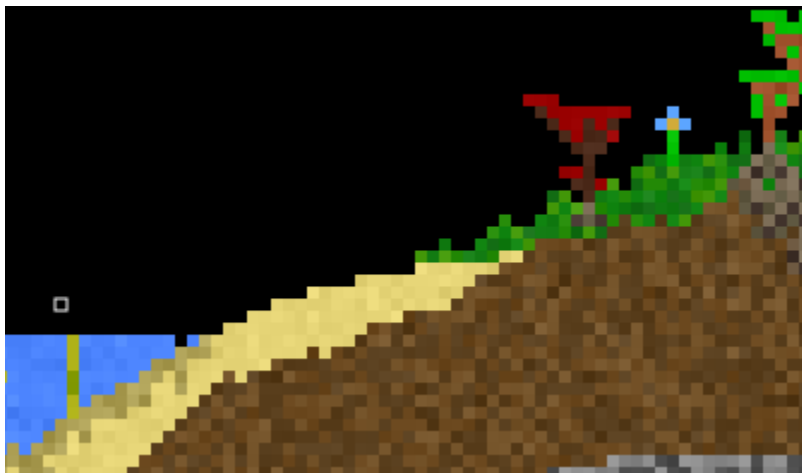
likes

- has a minimalist and clean ui
- elements have complex reactions to each other
- online sharing database
- has basic wind and pressure mechanics
- very good performance

dislikes

- limited number of elements
- unrealistic reactions
- repetitive elements
- fiddly place and erase tools

SandBoxels



<https://sandboxels.r74n.com/>

Overview

Sandboxels unlike Sandspiel takes a far more complex approach to simulating a sandbox conversely lots of it's feature make it less serious and more for a younger audience messing about.

There aren't many features I am likely to take from sandBoxels due to how unusual and high effort it is. I do like the temperature and gasses they have implemented but I think the gasses could have been implemented better which I will be trying to implement if I have the time. One of my biggest issues with SandBoxels is its lack of interactivity between elements so despite there being a large number of elements there isn't much which differentiates the elements without adding heat to fix this I will be creating far less elements in mine but focus heavily on details such as how they interact.

likes

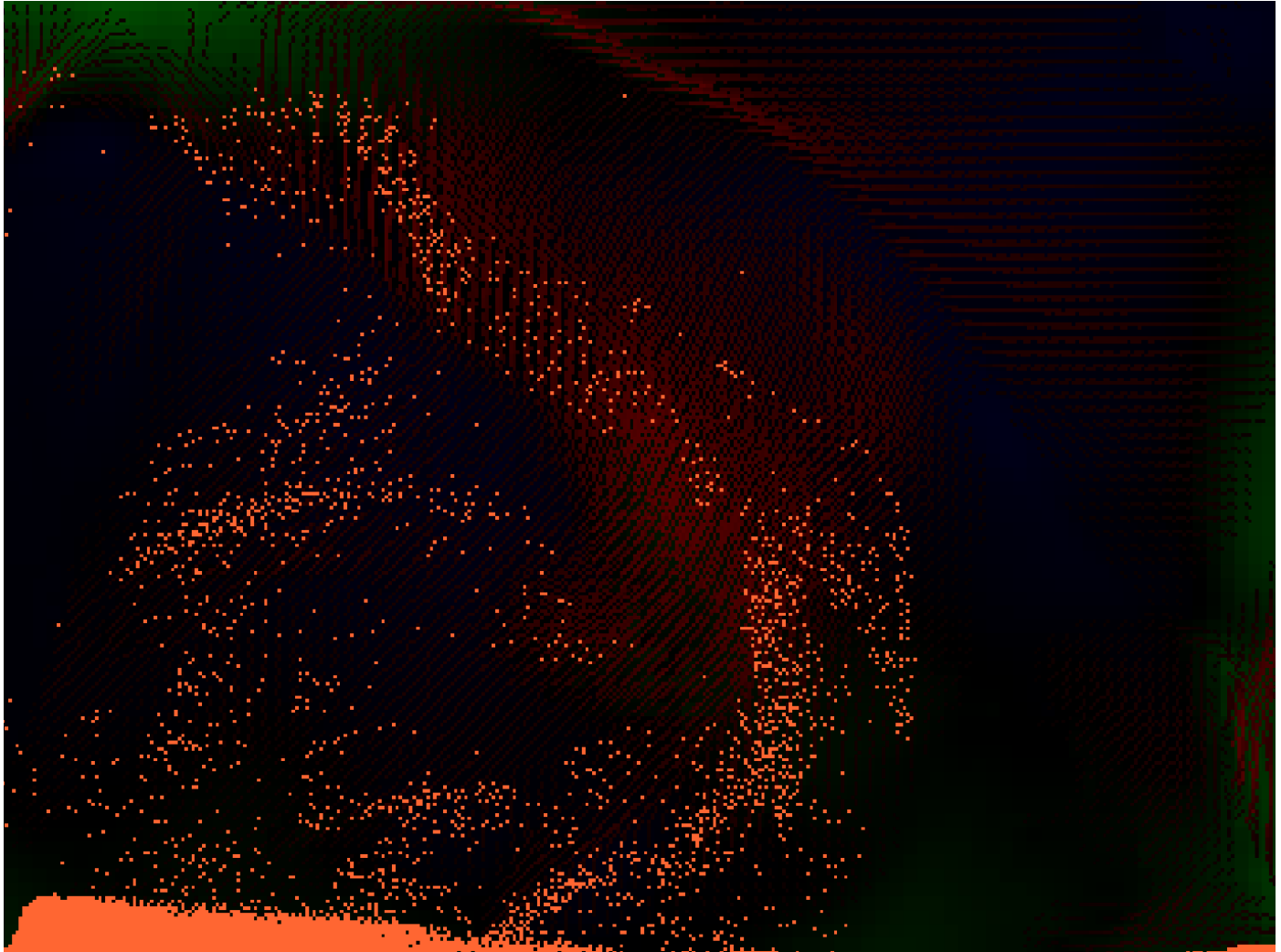
- has interesting states of matter and transitions of which
- has a couple interesting chemical reactions

- large number of elements
- has basic creature ai
- has interesting temperature mechanics

dislikes

- very few unique reactions
- very few mechanical processes
- very small working environment
- not a closed system so elements will disappear overtime

Powder Game



<https://dan-ball.jp/en/javagame/dust/>

overview

Powder Game is the oldest of these sandbox simulators and was the inspiration for sandspiel and was more a proof of concept than a fully optimised product but demonstrates some very interesting concepts.

As this was an early sandbox lots of its features have been refined and implemented in the previous examples, such as wind, tempurature/changing state. The wind mechanics are very complex with features like having lines indicating the winds direction and colouring the background based on wind pressure. One feature of Powder Game that I dislike is due to the year it was produced the number of particles you can

have is very limited to reduce memory usage, however this severely limits the scale of reactions you can build.

likes

- complex wind and pressure mechanics
- very unique reactions between the elements
- good selection of unique elements
- large community sharing creations

dislikes

- severely limited particle count
- cluttered ui
- temperature is rudimentary

Stakeholders

My stakeholders are people with an interests in science based subjects and who enjoy experimenting.

STEM students

My main demographic are people of all ages but primarily early A level(16-17 years old) who study STEM like subjects; primarily computer science, physics and engineering. A large majority of my players will likely play games as well with examples being besieged or Scrap Mechanic. According to statista there is little to no gender divide in gamers(1), There is also little gender divide in stem subjects as according [gov.uk](#)(2) in 2020 57% of the total stem A level students were male which is not a significant enough margin to tailor specifically either gender as well as it wouldn't affect my programme much.

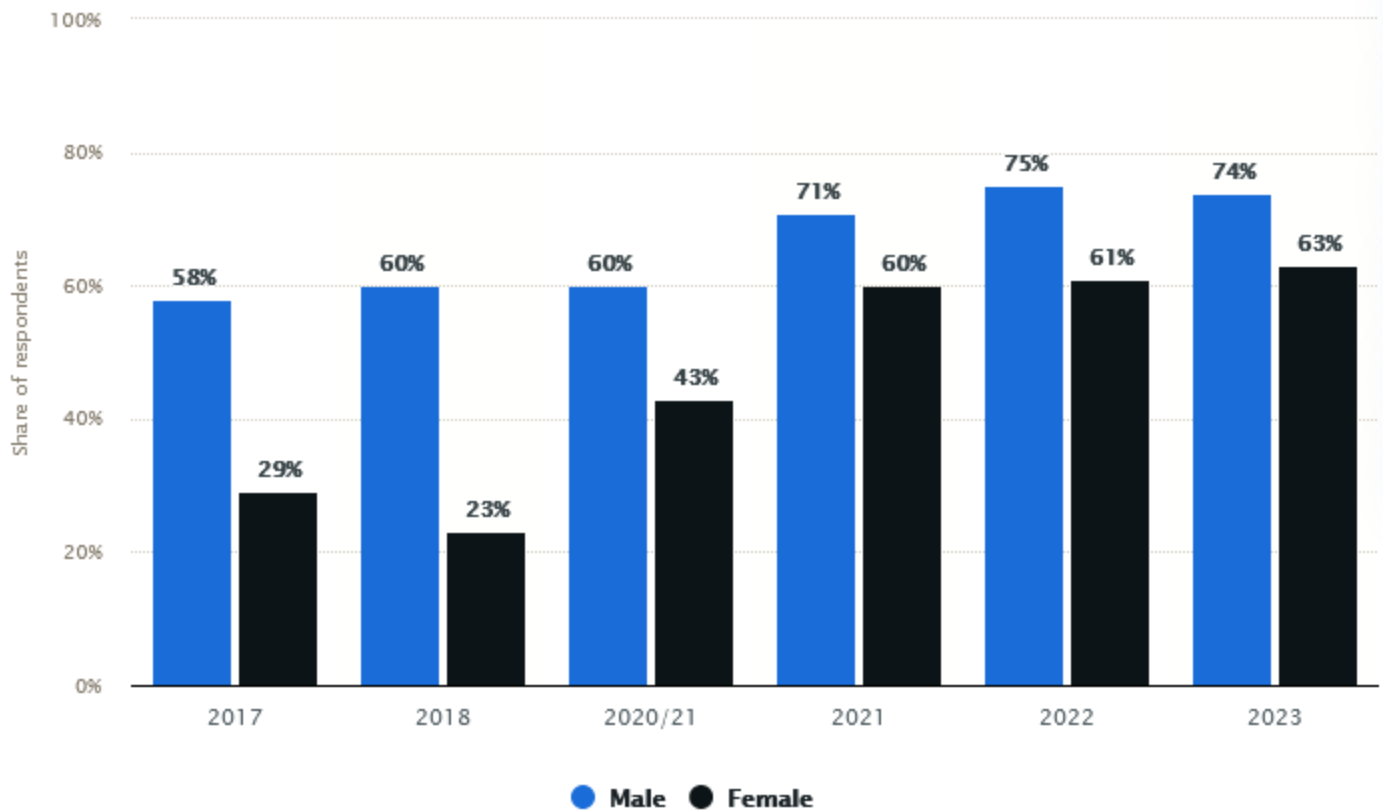
My programme will be appealing to this demographic as this game would allow them to try create highly complex and tailored reactions which will be aided by the highly precise tools I will make for this purpose making the overall experience highly enjoyable.

Secondary school students

While this is a smaller demographic I believe they are just as important as they will be where the majority of long time players come from, for this my programme will have to have some visually interesting simple reactions like explosions as well as some easy tools to use, this should lower the barrier of entry. I believe a sizable portion of these players will go on to be A level STEM students, my main demographic. This demographic will be mostly male students of between 13-16 years old.

My programme will be appealing to this demographic because from the surveys I conducted on this demographic they enjoyed watching large rapid reactions the most, which is very easy to accomplish in the simulation.

(1)



(2 left: total, middle: female applicants, right: male applicants)

All STEM subjects	Total STEM subjects	2020/21	267,856	117,076	150,779
		2019/20	255,928	112,572	143,354
		2018/19	258,782	113,371	145,411

Requirements

This is the order of which I will complete features with the more integral features coming first.

name	purpose	reason	importance
solid elements	the most basic element type, they will be effected by gravity, will act as walls and bases as well as having their own unique reactions.	to give the user a unique way to interact with the world that can lay as the foundation for other features	high
buttons	these are a core element of my project as they will allow the user to change; tools, elements, modes etc	the user needs to access the different tools and elements otherwise they would not be able to create anything meaningful	high
simulation viewer	the display will show the current state of the simulation and will	this will allow the user to know what is happening in the	high

name	purpose	reason	importance
	update every phrame	environment and allowing the user to get anything out of my project	
liquid elements	a simple element type, it will have basic fluid behaviours and will be more reactive and dynamic than solids	to give the user a more reactive element that will fill containers and crevices	high
placing and deleting elements	this will allow the user to edit the environment by placing and deleting different elements	this feature will make it so the programme is interactable which is necessary for this to be a game	high
reactions	this will allow the elements to effect eachother in interesting ways this is based on chemical reactions which may have other requiremnts	watching reactions is the main purpose of my project, without this feature there is no reason for the stakeholders to be interested	high
change of state	this will allow solids to melt, liquids to evaporate or freeze and gases to condensate	this will allow more interesting reactions allowing the user to create more realistic scenarios	medium
settings	this will allow the user to change keybindings, simulation speed and possibly other limitations	this is important as my shareholders will enjoy quality of life features like getting to use their custom configuration	medium
gaseous elements	these will not be affected by gravity but will rise in water or gravity affected solids and will be very reactive	this will allow the user to experiment with a more unstable element which has a different set of rules applied to it	medium
gas pressure	this will allow different gasses to overlap and have a repelling force away from eachother which can push liquids and solids	this is an interesting feature which will allow the user to model the flow of gasses more effectively	medium
temperature	this will add specific heat capacities and convection behaviours	this will allow the user to model thermal reactions more realistically and increases the chance of chain reactions	low
spawners, voids	spawners will add elements where as voids will destroy them	this allows a low level of automation which can be used to simulate a continuous environment which would be useful to my target demographic	low

hardware and software requirements

hardware

4gb ram

i3 or equivalent

128gb hhd

screen, mouse, keyboard

software

macos sonoma

jdk 21

Limitations

Lots of the features I listed likely won't be added due to scope issues and rather I will be focusing on more core gameplay features like gravity and element types.

clean ui

while not affecting the functionality of the game, a clean ui would add to the user experience and make the barrier to entry for learning how to use my project lower. I have decided to only do this if I have time as it would be massively time intensive for little payoff.

logic gates

Logic gates would be a really fun way for the user to interact with the programme allowing new ways to play especially for people interested in computer science which is listed as my main demographic in the Stakeholders section, however Integrating it with my program would take a significant amount of time and would likely need lots of complementary features to make it worth it.

mechanical

Mechanics such as pistons, rotaters, and jets would add a lot of interesting gameplay mechanics to the project and would be a feature my target audience would definitely enjoy however bug fixing it would be difficult as its a complex feature to add.

computational methods

My project is computationally suitable and would be near impossible to do otherwise for several reasons:

heuristics

My programme will not be simulating many minor effects and reactions, like the gravitational pull of each particle, and will only be focusing on the noticeable reactions, this is important as otherwise my programme would be incalculable in reasonable time.

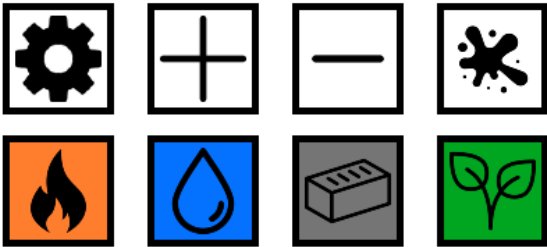
visualisation

Visualisation will make it possible for the user to understand as there is a gargantuan amount of data for each frame and it is important for the user to be able to understand what is happening and the game wouldn't be as enjoyable to my target audience without the visual flare, this is why a computer gui is necessary

concurrency

My programme will potentially have thousands of elements in the environment at once and each will have to check to see if dozens of conditions have been met and will have to react accordingly because of this I will use concurrency to check each element

UI and UX



Analysis:

Buttons

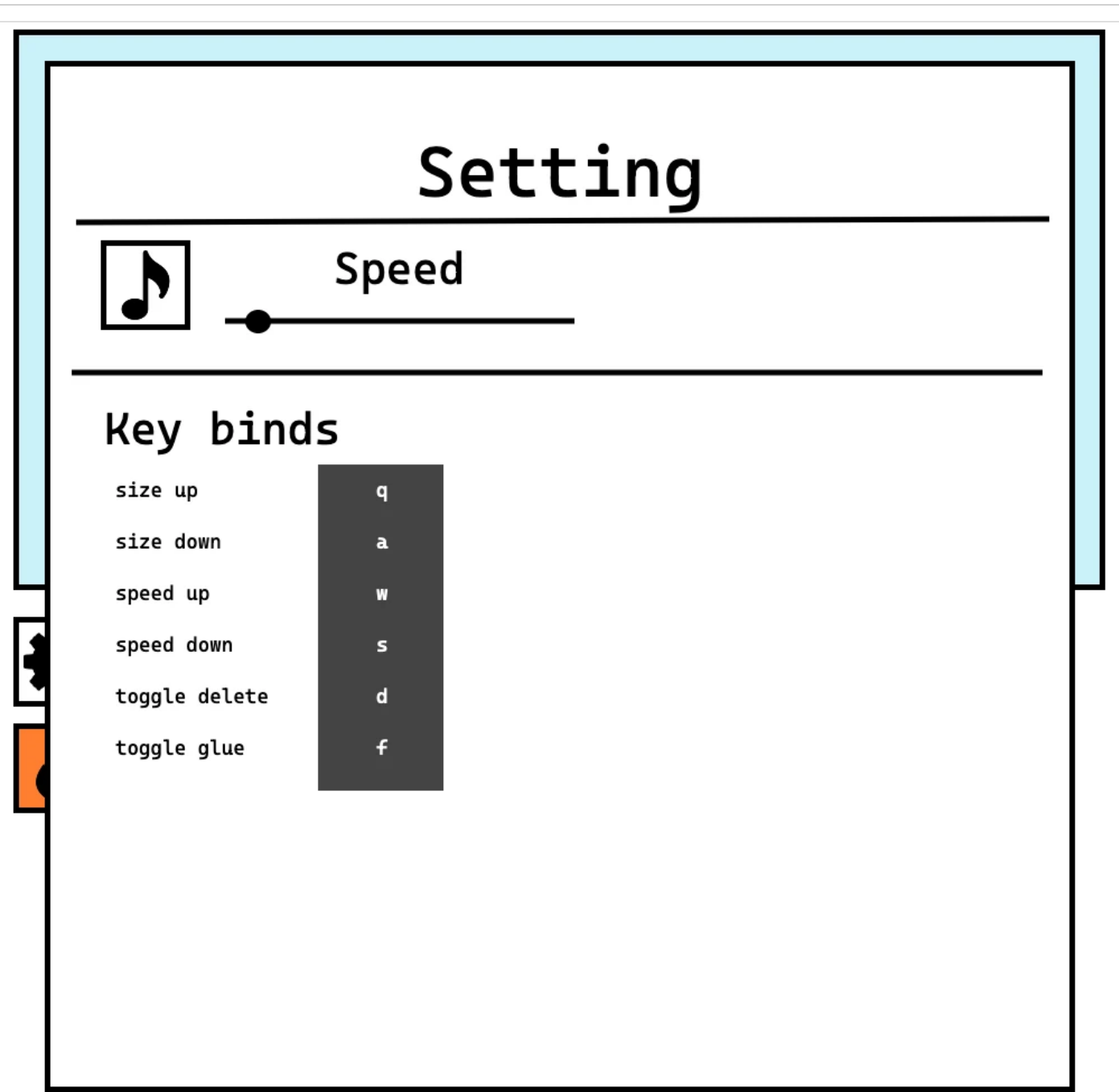
Each button has its own unique icon, I think this will be useful to the user because they are easier to understand than words and stand out better against the background. This is important as it improves the usability of the simulation and will reduce the barrier of entry to learning how my programme works.

Background

I have gone with a plain white background with a powder blue simulation background with thick black borders around places of interest, this will work well with my simulation as it needs to portray the information clearly and quickly so is improved by reducing visual clutter. This will also reduce the barrier to entry as there is less visual clutter making new players less confused.

Font

I went for an easily understandable clean font to match the theme I have gone for and has excellent readability.



Analysis

The settings menu is very important as my target audience is likely very tech literate and would benefit from larger customisation abilities.

layout

My settings menu will be divided into 3 parts, the top section just has the title of the page in large bold letters, this will be useful incase I add more pages as the title can make it easier to navigate. The second section is for commonly used settings, such as volume and simulation speed, this will allow quick access to

important features which is important as to not distract the user. Section three takes up the lower portion of the menu and houses the more technical elements of settings such as the keybinds for best customisation.

Development plan

The Development plan is a guide to help abstract the steps needed to complete my project.

UI implementation

planning

Before any simulation related development, I first need to create the UI, this will include a simulation window which will show what is happening in the simulation, I will also implement buttons in this stage.

It is important I fulfil this stage first as having an elegant UI is important to maintaining interest of my shareholders and by completing this step first I will spend more time with the UI and will be more likely to spot issues and allow early redesigns.

Features:

- simulation window
- feature buttons

implementation

I am using libGDX which is a library for high level interactions with openGL to manage my window and assets.

overview:

I had originally planned to have not many classes and instead have a smaller number of very broad classes as I thought this would make my code more maintainable however this led to many issues as some functions would call methods that work in the given object. To fix this I split Ulelement making a dedicated button class, which is a child of Ulelement, and introduced more type checking in my code.

In my plan I hadn't accounted for how the mouse would be handled in the code, in which I had to add a mouse class which reads the position from libgdx and then converts into my windows units

I also need to find new ways to store my Ulements as having them in an array has caused many issues.

testing

test number	test description	test data	expected result	actual result	fix
1.a	window launches	run programme	window opens and stays open	worked as expected	

test number	test description	test data	expected result	actual result	fix
1.b	buttons and window are in correct locations	run programme	buttons and window are orientated correctly	buttons and windows were upside down due to screen units being different from window units	flipped the y axis and shifted the scale

unexpected bugs:

buttons wouldn't unselect as Inheritance made many loopholes for detection of keypresses to be missed.
fixed by adding more typechecking\

element placing

Placing will be developed next because it will be necessary in order to test the later feature, this will include different size placement and different placement modes, some which will need deleting elements to work.

test number	test description	test data	expected result	actual result	fix
2.a	placing elements	pressing on the simulation view with placement selected	elements get added to the simulation		
2.b	changing add tool size	adding elements to simulation view	number of elements being added fits setting		

element removing and replacing

Removing elements

possible changes: due to me loving the style of the settings menu I may edit the main menu to be similar

visualisation: there will be visualisation to help the user understand whats happening // had an idea for order of checking reactions that you do it in order of reactivity // reorientate towards it being more leisure based than realistic based